

Deep SARSA vs Advantage Actor-Critic

A Comparative Study on LunarLander-v3

Francesco Galardi — Andrea Vagnoli

Symbolic and Evolutionary Artificial Intelligence
University of Pisa — AIDE Master's Programme

June 2026

Outline

Environment & MDP

- LunarLander-v3 overview
- State & Action spaces
- Reward function & stochasticity

Implementation

- Network architecture
- Training setup & hyperparameters
- Exploration strategies

Algorithms

- SARSA: theory & neural extension
- Actor-Critic (A2C): theory & design
- Side-by-side comparison

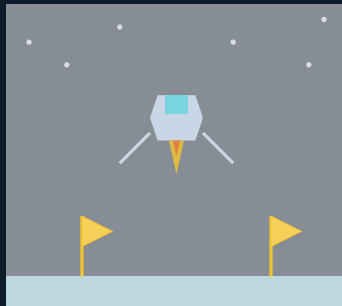
Results

- Learning curves (mean $\pm$ 95% CI)
- Statistical significance (Welch's t -test)
- Efficiency, generalisation, failure analysis

The LunarLander-v3 Environment

Setting: A rocket must land safely between two flags.

- Simulates realistic gravitational physics
- Main engine + two lateral thrusters
- Episode ends on landing, crash, or out-of-bounds
- **Solved** when mean reward ≥ 200 over 100 episodes



Why LunarLander-v3?

- **Continuous state, discrete actions:** ideal for comparing value-based vs. policy-gradient methods
- Well-known benchmark with clear success

MDP Formulation: State & Action Space

State Space $\mathcal{S} \subset \mathbb{R}^8$

Index	Feature	Range
s_0	x position	$[-1.5, +1.5]$
s_1	y position	$[0, +1.5]$
s_2	x velocity	$\mathbb{R}$
s_3	y velocity	$\mathbb{R}$
s_4	angle θ	$[-\pi, +\pi]$
s_5	angular velocity	$\mathbb{R}$
s_6	left leg contact	$\{0, 1\}$
s_7	right leg contact	$\{0, 1\}$

Normalisation: running mean/std (Welford) applied per-observation to keep inputs well-conditioned for both networks.

Action Space $\mathcal{A}$ — Discrete(4)

Action	Effect
0	Do nothing
1	Fire left thruster
2	Fire main engine
3	Fire right thruster

Impact on algorithm choice

Discrete actions allow **Deep SARSA** to output Q-values for all 4 actions in one forward pass, and allow **A2C** to use a *Categorical* distribution with clean entropy computation $H = -\sum_a \pi \log \pi$.

MDP Formulation: Reward & Stochasticity

Reward Function (Dense)

Per-step shaping reward:

$$r_t = \underbrace{-\Delta d}_{\text{approach}} + \underbrace{-0.03|v|}_{\text{velocity pen.}} + \underbrace{-0.3|\omega|}_{\text{rotation}} + \underbrace{0.1 \cdot (\ell_L + \ell_R)}_{\text{leg contacts}}$$

Plus terminal bonuses:

- +100 for a successful landing
- -100 for crash or boundary exit
- -0.3 per engine firing (fuel cost)

Dense reward: feedback at every step reduces the credit assignment problem compared to sparse alternatives.

Stochasticity Sources

- **Initial conditions:** position and velocity sampled from a bounded uniform distribution
- **Optional wind:** $\mathcal{N}(0, \sigma_w^2)$ lateral force added each step
- **Turbulence:** random torque $\mathcal{N}(0, \sigma_T^2)$

Reward hacking risk

Agent may learn to *hover* without landing to avoid the crash penalty. Monitored via episode length and leg-contact rate

Algorithm 1: SARSA — Theory

SARSA (Sutton & Barto, 2018, §6.4) is a model-free, *on-policy* TD control algorithm.

At each step the agent uses the tuple $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ to update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

- a_{t+1} is sampled from the **current policy** (ϵ -greedy) — not from $\max_{a'}$
- This makes the method **on-policy**: the target includes the next action actually taken
- Converges to the optimal policy under mild conditions

SARSA vs Q-Learning

	SARSA	Q-Learning
Target	$Q(s', a')$	$\max_{a'} Q(s', a')$
Policy	on-policy	off-policy
Safety	safer	riskier

Key intuition

SARSA is *pessimistic*: it evaluates the policy it actually follows, including exploration mistakes. This yields more **conservative** policies in stochastic environments.

Deep SARSA: Neural Network Extension

Tabular SARSA cannot handle the continuous state space ($\mathcal{S} \subset \mathbb{R}^8$). We replace the Q-table with a neural network $Q_\theta(s, \cdot)$: one forward pass yields Q-values for *all* 4 actions simultaneously.

Loss function:

$$\mathcal{L}(\theta) = \left(Q_\theta(s, a) - \underbrace{[r + \gamma Q_{\theta^-}(s', a')]}_{\text{SARSA target}} \right)^2$$

where θ^- are the *target network* parameters (lagged copy, synced every 25 episodes).

Extensions over base codebase

- **Target network:** stabilises the Bellman target, preventing oscillations (Mnih et al., 2015). Synced every 25 episodes.
- **Short-horizon replay buffer (50 k):** de-correlates consecutive gradient updates while staying *approximately* on-policy (stores a' alongside each transition)
- **Gradient clipping** ($\|\nabla\| \leq 10$) for stability in early training

On-policy integrity

Algorithm 2: Advantage Actor-Critic (A2C) — Theory

A2C is a synchronous *policy-gradient* method (Mnih et al., 2016) combining:

- **Actor** $\pi_\theta(a|s)$: stochastic policy
- **Critic** $V_w(s)$: state-value baseline

Actor gradient (REINFORCE with baseline):

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot A(s, a)]$$

n -step advantage ($n = 200$, near-Monte Carlo):

$$A_t = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_w(s_{t+n}) - V_w(s_t)$$

Critic loss: $\mathcal{L}_c = (G_t - V_w(s_t))^2$

Entropy regularisation

$$\mathcal{L}_\pi = -\log \pi_\theta(a|s) \cdot A_t - \beta H[\pi_\theta(\cdot|s)]$$

The entropy bonus $\beta H[\pi]$ prevents premature convergence to deterministic (brittle) policies. Final value: $\beta = 0.001$.

Why A2C over plain REINFORCE?

- Critic reduces gradient variance via the baseline
- n -step returns balance bias–variance

Algorithm Comparison

Property	Deep SARSA	A2C
Paradigm	Value-based (TD)	Policy gradient
Policy type	Implicit (ε -greedy on Q)	Explicit (π_θ)
On/Off policy	On-policy	On-policy
Exploration	ε -greedy	Entropy bonus $\beta H[\pi]$
Replay buffer	Yes (50 k, semi-on-policy)	No
Target network	Yes (sync every 25 ep)	No
Networks	1 Q-network + target	Actor + Critic (separate LR)
Update frequency	Per mini-batch (batch=64)	Every $n = 200$ steps
Training budget	2 000 episodes	5 000 episodes

A2C advantage

Neural Network Architecture

Deep SARSA — single Q-network (+ frozen target copy):

$$\mathbb{R}^8 \xrightarrow{\text{FC-256}} \text{ReLU} \xrightarrow{\text{FC-256}} \text{ReLU} \xrightarrow{\text{FC-4}} \mathbb{R}^4$$

Output: $Q(s, a)$ for each of the 4 actions.

A2C — two separate networks:

$$\underbrace{\mathbb{R}^8 \rightarrow 256 \rightarrow 256 \rightarrow 4}_{\text{Actor: Softmax} \Rightarrow \pi_{\theta}(a|s)} \quad \underbrace{\mathbb{R}^8 \rightarrow 256 \rightarrow 256 \rightarrow 1}_{\text{Critic: } V_w(s)}$$

Design choices

- **Width 256:** matches RL Baselines3 Zoo tuned architecture for LunarLander-v3 (Raffin et al., 2021)
- **ReLU:** avoids vanishing gradients; standard for discrete-action control tasks
- **Separate actor/critic:** independent learning rates ($\alpha_{\pi} = 3 \times 10^{-4}$, $\alpha_V = 10^{-4}$) reduce the risk of the critic destabilising the actor
- **Gradient clipping:** $\|\nabla\| \leq 10$ (SARSA Q-net), $\|\nabla\| \leq 0.5$ (A2C actor & critic)

Training Setup & Hyperparameters

Deep SARSA (2 000 episodes)

Parameter	Value
Learning rate α	3×10^{-4}
Discount γ	0.99
$\varepsilon_0 \rightarrow \varepsilon_\infty$	$1.0 \rightarrow 0.01$
ε decay	0.9975 / ep
Target sync freq.	25 episodes
Batch size	64
Buffer capacity	50,000

Actor-Critic A2C (5 000 episodes)

Parameter	Value
Actor LR α_π	3×10^{-4}
Critic LR α_v	1×10^{-4}
Discount γ	0.99
Entropy coef β	0.001
n -step horizon	200
Grad clip (actor/critic)	0.5

Common setup & hyperparameter source

5 seeds {42, 123, 456, 789, 1234}, observation normalisation (Welford). Initial values from **RL Baselines3 Zoo** (Raffin et al., 2021) — Optuna-tuned on LunarLander-v3. Final values adjusted from training logs (buffer 10k $\rightarrow$ 50k; β 0.05 $\rightarrow$ 0.001; n_{steps} 5 $\rightarrow$ 200).

Exploration–Exploitation Trade-off

Deep SARSA — ε -greedy

$$\pi(a|s) = \begin{cases} \text{Uniform}(\mathcal{A}) & \text{w.p. } \varepsilon_t \\ \arg \max_a Q_\theta(s, a) & \text{w.p. } 1 - \varepsilon_t \end{cases}$$

$\varepsilon_t = \max(0.01, 0.9975^t)$. Reaches 0.01 at $\text{ep} \approx 1840$

(from $0.9975^n = 0.01 \Rightarrow n \approx 1840$).

Pros: simple, analytically calibrated to budget.

Cons: exploration is *undirected* (uniform random).

A2C — Entropy Regularisation

$$H[\pi] = - \sum_a \pi_\theta(a|s) \log \pi_\theta(a|s)$$

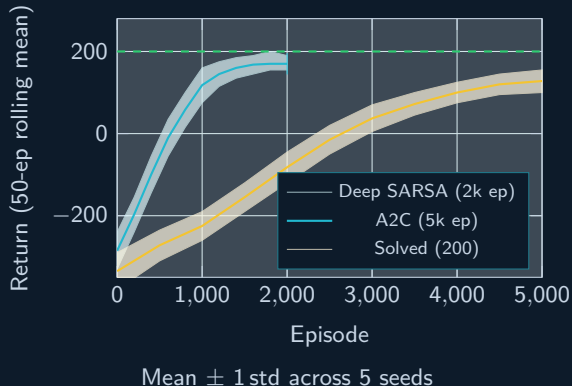
With $\beta = 0.001$, entropy stays at $H \approx 0.87\text{--}0.95$ for all 5 000 episodes — policy remains stochastic throughout.

Pros: exploration adapts to policy uncertainty; aids zero-shot transfer to harder variants.

Cons: β critical and hard to tune; policy never fully commits, slowing convergence.

Key insight: ε -greedy stops exploring at $\text{ep} \approx 1840$; A2C's persistent stochasticity costs convergence speed but gains deployment robustness.

Results: Learning Curves



Observations

- **SARSA** converges by ep ~ 1100 ; final mean 169.8 ± 15.6 in only 2 000 episodes
- **A2C** needs $\sim 3\times$ more episodes; 4/5 seeds converge; final mean 127.5 ± 31.1
- **Lower cross-seed variance** for SARSA (std 15.6 vs 31.1) — more predictable training
- Difference **not statistically significant** on standard env ($p = 0.052$)

Statistical Analysis

Method: Welch's t -test on mean return over the last 100 episodes, averaged over 5 seeds.

Hypotheses:

- $H_0: \mu_{\text{SARSA}} = \mu_{\text{A2C}}$
- $H_1: \mu_{\text{SARSA}} \neq \mu_{\text{A2C}}$

Metric	SARSA	A2C
Mean return	169.83	127.48
Std (5 seeds)	15.64	31.14
95% CI	[150.4, 189.3]	[88.8, 166.1]
t -statistic	2.43	
p -value	0.052	
Verdict	Cannot reject H_0 ($p \geq 0.05$)	

Why Welch's t -test?

- Handles **unequal variances** (Welch, 1947) — appropriate when $\sigma_S \neq \sigma_A$ (15.6 vs 31.1)
- With $n = 5$ seeds the test has limited statistical power: small samples inflate p -values

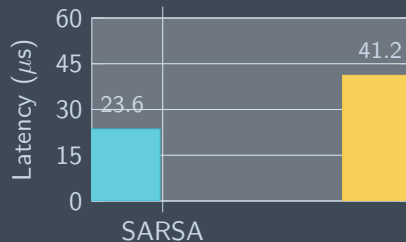
Interpretation

Despite SARSA's higher mean (+42 pts), CIs overlap and $p = 0.052$ is just outside significance. **Both agents are competitive** on standard. The real

Training complexity

- **Deep SARSA**: mini-batch (batch=64) every step once buffer warm; buffer fills by ep ~ 330 ; overhead from replay sampling + target sync every 25 ep
- **A2C**: gradient update every $n = 200$ steps; two network forward/backward passes; no buffer overhead
- **Sample efficiency**: SARSA needs 2 000 ep vs A2C 5 000 ep ($2.5\times$ fewer environment interactions to reach comparable performance)

Inference latency (CPU, $N = 1000$ calls)



SARSA 0.024 ms (single Q-net forward).

A2C 0.041 ms (actor forward + Categorical sample).

A2C is $1.75\times$ slower at inference.

Generalisation: Environment Variants

Agents trained on *standard* are evaluated **zero-shot** on three harder variants (seed 42 checkpoint, 20 greedy episodes):

Variant	SARSA	A2C
Standard	+130 $\pm$ 171	+163 $\pm$ 117
Wind ($\sigma_w=15$)	+19 $\pm$ 165	+102 $\pm$ 132
Turbulent (wind+torque)	-40 $\pm$ 162	+109 $\pm$ 127
Heavy ($g=-15$)	+137 $\pm$ 103	+160 $\pm$ 105

Mean $\pm$ std, 20 episodes per variant.

Key findings

- **A2C generalises far better:** turbulence +109 vs SARSA -40 — a **149-point gap**
- **SARSA collapses under perturbation:** greedy policy fitted to no-wind dynamics; -85% drop from standard to turbulent
- **A2C robust:** entropy-regularised stochastic policy retains $\sim 65\%$ of standard performance across all variants
- **Heavy gravity** manageable for both: same control structure, smoother

Deep SARSA failure modes

- **Bimodal behaviour:** within-episode std ≈ 150 –165 (min -585) — agent either lands or crashes; structural, not reducible by more training episodes
- **Brittleness under perturbation:** greedy policy ($\epsilon \rightarrow 0.01$) exploits exact no-wind dynamics; wind $+19$, turbulent -40
- **Reliable across seeds:** std 15.6 vs A2C 31.1; all 5 seeds converge ep 1050–1250

A2C failure modes

- **Policy stagnation:** entropy stays flat at $H \approx 0.87$ –0.95 throughout; 1/5 seeds never reaches mean 200
- **Slow, seed-sensitive convergence:** ep 1880–4580 to converge; $4\times$ wider CI than SARSA
- **Critic sensitivity:** $\alpha_V = 10^{-3}$ caused divergence (loss > 3000); required 10^{-4}

Shared limitation

Both are **on-policy**: data cannot be reused across policy updates, limiting sample efficiency vs. off-policy methods (DQN, SAC). The generalisation gap could be closed by fine-tuning on each

Hyperparameter Selection Rationale

Starting point: RL Baselines3 Zoo (Raffin et al., 2021)

Values optimised via **Optuna** on LunarLander-v3. Our structural choices match the Zoo:

- Buffer: 50,000 ✓ (DQN config)
- Architecture: [256, 256] ✓ (both)
- $\gamma = 0.99$ ✓ (both)

SARSA adjustments from training logs

- α : $5 \rightarrow 3 \times 10^{-4}$ (Q oscillations)

A2C path to final configuration

β	n_{steps}	Outcome
0.05	5	Max entropy; no learning
0.01	5	Critic diverges (loss > 500)
0.02	20	Stable critic, no convergence
0.02	200	First convergence ($\sim$ ep 3000)
0.001	200	Final — consistent

Episode budget rationale

SARSA: $0.9975^{1840} \approx 0.01$ — budget covers the full exploration curve + ~ 160 exploitation ep.

A2C: empirical — most seeds converging

Conclusions

Summary

- Implemented and compared two fundamentally different RL paradigms on LunarLander-v3, 5 seeds each
- **Deep SARSA**: faster convergence (ep ~ 1100), higher training performance (169.8 ± 15.6), lower cross-seed variance — but *brittle* to perturbation
- **A2C**: slower and more seed-sensitive (127.5 ± 31.1), yet **far more robust**: turbulent +109 vs SARSA -40
- Difference **not statistically significant** on standard env ($p = 0.052$); real gap on variants
- **Trade-off**: SARSA for fast reliable training;

Future directions

- n -step SARSA / eligibility traces
- PPO or SAC for comparison
- Multi-seed eval on variants
- Hyperparameter search via Optuna
- Fine-tuning on wind/turbulent variants
- Continuous action space (LunarLanderContinuous-v3)

Key takeaway

References

-  R.S. Sutton & A.G. Barto (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press. §6.4 (SARSA), §13 (Policy Gradient).
-  V. Mnih et al. (2015). *Human-level control through deep reinforcement learning*. Nature 518, 529–533.
-  V. Mnih et al. (2016). *Asynchronous Methods for Deep Reinforcement Learning*. ICML 2016.
-  P. Henderson et al. (2018). *Deep Reinforcement Learning that Matters*. AAAI 2018.
-  G. Brockman et al. (2016). *OpenAI Gym*. arXiv:1606.01540.
-  A. Raffin et al. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. JMLR 22(268). <https://github.com/DLR-RM/rl-baselines3-zoo>
-  JohDonald (2021). *Deep-Q-Learning-Deep-SARSA-LunarLander-v3*. <https://github.com/JohDonald/Deep-Q-Learning-Deep-SARSA-LunarLander-v2>
-  N. Barhate (2021). *Actor-Critic-PyTorch*. <https://github.com/nikhilbarhate99/Actor-Critic-PyTorch>